

Tasks:

For the given system, please answer the following questions:

1. In which Security zone do the both parts (PLC and SCADA) run? Which part is considered as a lower trust level? (Points: 5)

In this lab, the PLC is implemented using the OpenPLC runtime, which directly executes the control logic of the water heater and interacts with the physical process. According to the ICS architectural model, the PLC operates at Level 1 and belongs to the Cell Security Zone, which contains critical control components with direct influence on the physical process.

The SCADA system is implemented using ScadaBR, which provides a graphical interface for monitoring the process and for changing operational parameters such as setpoint and control mode. The SCADA operates at Level 3 and belongs to the Manufacturing (Supervisory) Security Zone, which is responsible for supervision and monitoring rather than direct control execution.

Regarding trust levels, the SCADA/HMI (ScadaBR) is considered the lower-trust component. It is user-facing, runs on a general-purpose software stack, and communicates with the PLC via Modbus/TCP, a protocol that does not provide authentication or encryption. As a result, the SCADA represents a more exposed attack surface compared to the PLC.

2. Please perform a real MitM attack against the given system e.g., the attacker places himself in the middle between the OpenPLC and the HMI (ScadaBR), and listens to all the packet transferred between the stations (see Figure 5). In addition, the attacker maliciously alters the packet he receives and sends it back to the other station in order to confuse the physical process. (Points: 15)

In order to demonstrate the **passive phase of a MitM attack**, the attacker monitored the communication between the SCADA/HMI (ScadaBR) and the PLC (OpenPLC).

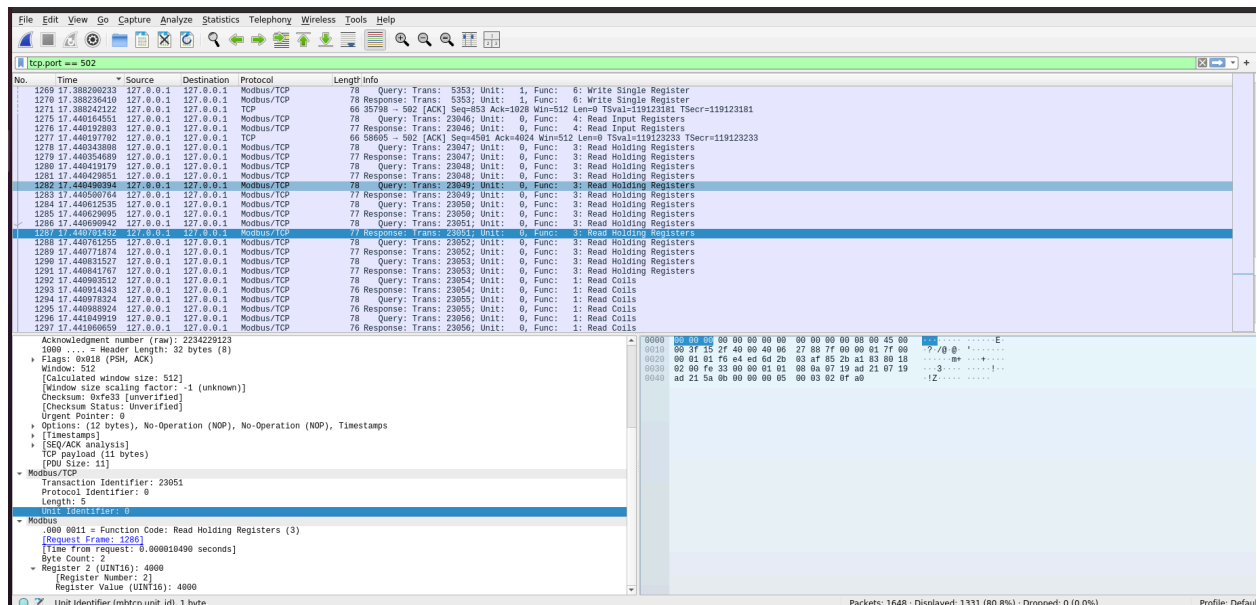
Since both components run on the same virtual machine, the traffic was captured on the loopback interface.

Wireshark was used as a network sniffer to observe the exchanged packets. A display filter was applied to restrict the captured traffic to Modbus/TCP communication by filtering on TCP port 502 (tcp.port == 502), which is the standard port used by Modbus/TCP.

While interacting with the HMI (e.g., changing the setpoint), Modbus/TCP request and response messages were observed. In particular, Read Holding Registers (Function Code 3) responses contained raw register values transmitted by the PLC. By correlating these register values with the values shown in the HMI and the PLC monitoring interface, the attacker was able to infer the transmitted process data.

For example, a captured Modbus/TCP response contained a register value of 4000, which corresponds to a setpoint of 40.00°C as displayed in the HMI and OpenPLC monitoring interface. This demonstrates that an attacker is able to listen to and interpret the control traffic exchanged between the SCADA system and the PLC.

The following image shows a Wireshark capture of Modbus/TCP traffic on port 502, including a Read Holding Registers response containing the setpoint value.



After demonstrating the passive Man-in-the-Middle attack, an **active MitM attack** was performed by placing an attacker-controlled proxy between the SCADA/HMI (ScadaBR) and the PLC (OpenPLC), as illustrated in Figure 5 of the exercise sheet.

Since both components run on the same virtual machine, the attacker redirected the SCADA communication to the proxy by changing the Modbus/TCP port in ScadaBR from the default port 502 to port **1502**. The proxy listens on this port and forwards traffic to the real PLC on port 502, allowing the attacker to intercept all exchanged Modbus/TCP packets.

A Python-based MitM script (**attack_task2.py**) was executed to relay traffic bidirectionally between ScadaBR and OpenPLC. In contrast to the passive attack, the script actively modifies packets sent from the PLC to the SCADA system. Specifically, a payload byte within Modbus/TCP response messages was altered before being forwarded to the HMI.

The successful manipulation of packet contents was confirmed through attacker-side logs, which show the original and modified byte values (0x00 changed to 0xFF) during transmission. Because the communication is proxied locally over the loopback interface, Wireshark captures the original PLC packets before modification. For this reason, attacker-side logs are provided as evidence, demonstrating that packet contents are altered in memory prior to being forwarded to the SCADA system.

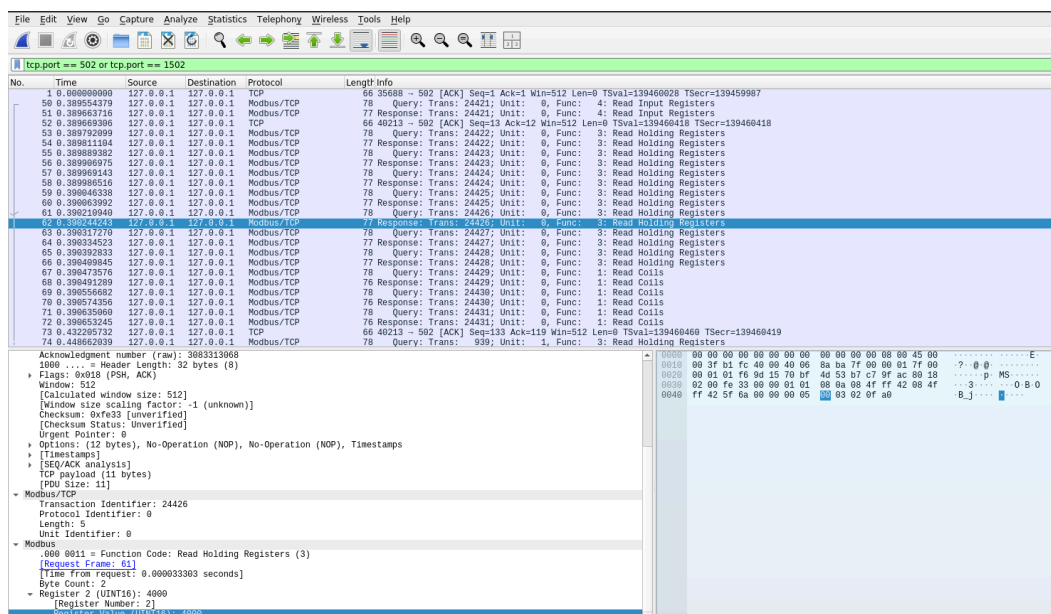
```
csaa@CSAA-final:~/Downloads$ sudo python3 attack_task2.py
[sudo] password for csaa:
[+] MITM running on port 1502
[+] ScadaBR connected
[SCADA -> PLC] 12 bytes
[PLC -> SCADA] 12 bytes
Byte 10 before: 00
Byte 10 after:  ff
DATA MODIFIED!
```

This experiment demonstrates that an attacker positioned in the network can not only observe but also actively manipulate industrial control traffic, highlighting the lack of integrity protection in Modbus/TCP communication.

3. Please perform a False Data Injection attack where the attacker sends crafted packets to the target PLC as shown in Figure 6. The attack should change the PLC registers' values, inputs, outputs, etc. At the same time, the user on HMI should not be able to catch your attack. (Points: 20)

To perform the False Data Injection (FDI) attack, the attacker directly sent crafted Modbus/TCP packets to the OpenPLC without intercepting the communication between the PLC and the HMI.

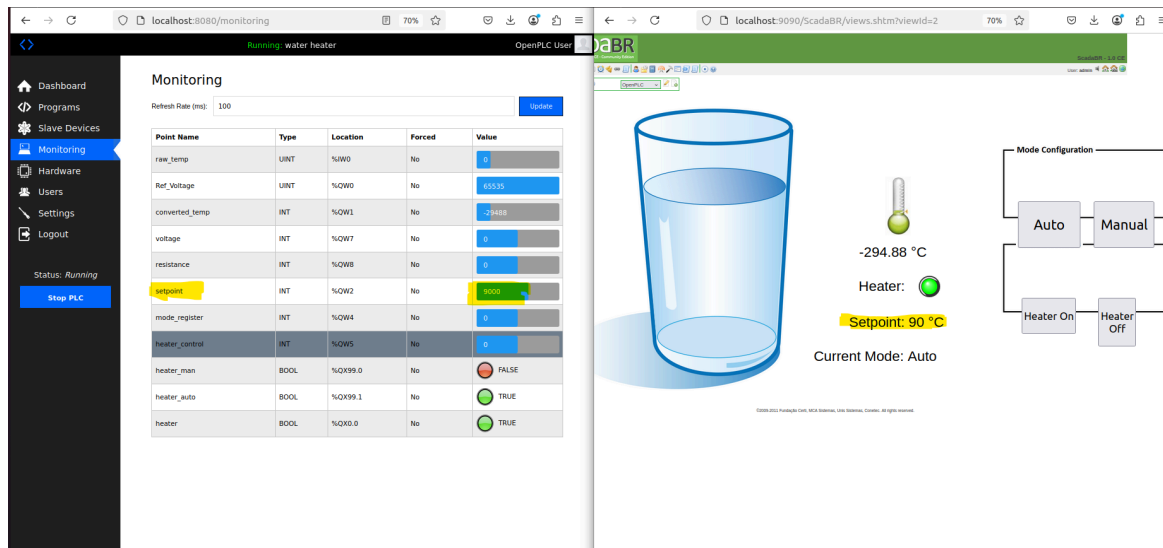
First, a Wireshark analysis of the Modbus/TCP traffic was carried out in order to identify the register responsible for storing the setpoint value. By inspecting Read Holding Registers (Function Code 3) responses exchanged between ScadaBR and OpenPLC, it was observed that the setpoint value is stored in holding register 2, which can be identified by correlating the returned register values with the setpoint displayed on the HMI.



Based on this information, a Python script (**attack_task3.py**) was developed using the *pymodbus* library. The script establishes a direct Modbus/TCP connection to the PLC on port 502 and periodically injects falsified values into holding register 2. In this case, a fake setpoint value of 90 °C (scaled as 9000) is written every five seconds, without any interaction from the HMI.

```
csaa@CSAA-final:~/Downloads$ python3 attack_task3.py
[+] Connected to OpenPLC
Injected fake temperature: 90.0°C
Injected fake temperature: 90.0°C
Injected fake temperature: 90.0°C
```

As a result of the attack, the setpoint value changes automatically in the OpenPLC monitoring interface and is also reflected in the ScadaBR dashboard. From the HMI perspective, the system continues to operate normally and no error or alert is displayed, making the attack invisible to the user.



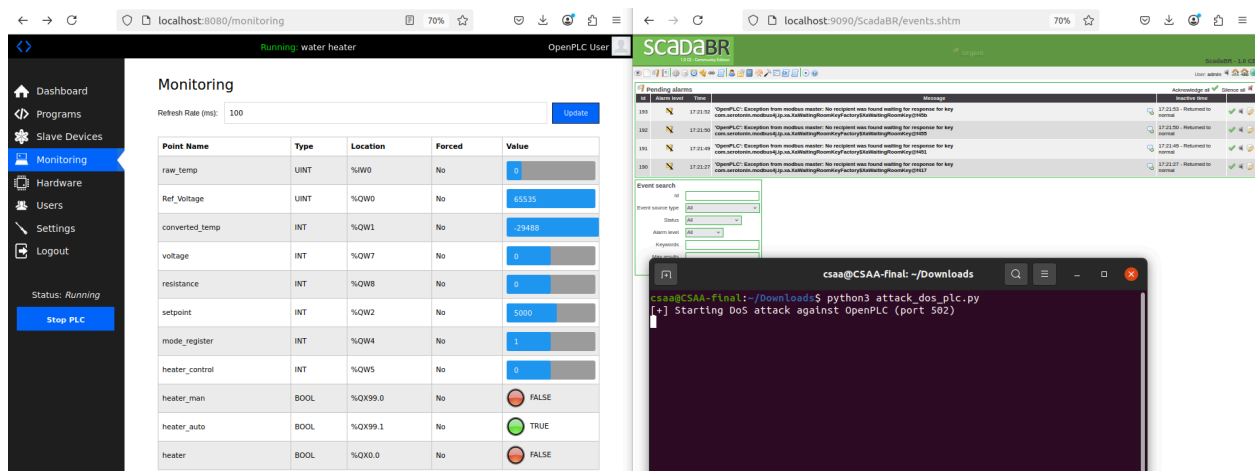
4. Please perform a flooding attack i.e. Denial of Service (DoS) attack against both HMI and PLC (see Figure 7). The attack should make the PLC/HMI out of the service i.e., the victim is unable to reply to legitimate requests from the user. (Points: 10)

To perform a Denial of Service (DoS) attack against the PLC, a flooding-based approach was implemented targeting the Modbus/TCP service exposed by OpenPLC on port 502. A Python script (**attack_dos_plc.py**) was developed using the socket and threading libraries in order to generate a large number of concurrent TCP connections to the PLC.

The script continuously opens multiple TCP connections to the PLC and sends arbitrary data while keeping the connections active. By launching several parallel threads, the number of simultaneous connections rapidly increases, exhausting the PLC communication resources and preventing it from responding to legitimate Modbus requests from the HMI.

During the execution of the attack, the OpenPLC web interface became significantly slow and the monitoring view frequently froze, indicating that the PLC was no longer able to process requests in a timely manner. At the same time, ScadaBR reported multiple Modbus communication warnings, such as missing or delayed responses from the PLC. These warnings confirm that the HMI was unable to retrieve updated process data while the attack was ongoing.

Although the PLC process remained in a running state, legitimate communication between ScadaBR and OpenPLC was effectively disrupted. A screenshot showing the unresponsive OpenPLC monitoring interface and the corresponding warning messages in ScadaBR is attached below.



To perform a DoS attack against the HMI, a flooding-based attack was launched targeting the ScadaBR web interface, which is exposed via an HTTP service running on port 9090. A Python script (**attack_dos_hmi.py**) was developed using the *socket*, *threading*, and *time* libraries in order to generate a large number of concurrent HTTP requests to the ScadaBR server.

The script establishes multiple TCP connections to the ScadaBR service and repeatedly sends HTTP GET requests to the /ScadaBR/ endpoint using persistent connections (keep-alive). By sending many requests over the same connection and running several threads in parallel, the web server resources are rapidly exhausted. The connections are intentionally kept open for a short period of time to increase resource consumption before being closed.

During the execution of the attack, the ScadaBR web interface became extremely slow and eventually unresponsive. When accessing the HMI through the browser, the page remained stuck in a loading state and the graphical elements of the interface (such as the process visualization and control diagrams) were not rendered.

In some cases, the server returned an HTTP 500 Internal Server Error indicating a *java.lang.OutOfMemoryError: GC overhead limit exceeded*. This error confirms that the web server ran out of available resources due to the high volume of concurrent requests generated by the flooding attack, preventing ScadaBR from properly handling legitimate user requests.

Screenshots showing both the frozen loading interface and the internal server error are attached as proof of concept, demonstrating that the HMI was effectively unavailable during the DoS attack.

